Atividade realizada em 17/08/2026

Notebook Aula 02: Representação de Dados e Estruturas em Python (Profª Kadidja)

Tópico 1: Fundamentos — Dados na Memória e Referências (`id()`)

Em Python, as variáveis funcionam como **referências** para objetos alocados na memória RAM. A função nativa `id()` permite observar o identificador único associado a esse objeto durante sua existência.

```
# ================================================================================
# TÓPICO 1: Dados na Memória
# Demonstração de referências de variáveis e busca do ID único na memória RAM
# ================================================================================

# Criação de um objeto do tipo inteiro (20) e atribuição à variável 'idade'
idade = 20

# Exibe o valor contido e o endereço único de memória atribuído pelo Python
# print(f"Valor inicial da variável 'idade': {idade}")
print("Valor inicial da variável 'idade'", idade)
print(f"Endereço/ID na memória RAM (id(idade)): {id(idade)}")

# Ao reatribuir um novo valor, Python cria um novo objeto na memória
idade = 21
print(f"\nNovo valor atribuído a 'idade': {idade}")
print(f"Novo endereço/ID na memória RAM: {id(idade)}")
```

```
Valor inicial da variável 'idade' 20
Endereço/ID na memória RAM (id(idade)): 11645960

Novo valor atribuído a 'idade': 21
Novo endereço/ID na memória RAM: 11645992
```

Tópico 2: Estruturas Sequenciais — Listas e Indexação

A **Lista (`list`)** é a principal estrutura sequencial em Python, substituindo os vetores/arrays tradicionais. O acesso ocorre por **índices positivos** (a partir de `0`) e por **índices negativos** para navegação reversa (sendo `-1` o último elemento).

```
# ================================================================================
# TÓPICO 2: Estruturas Sequenciais (Listas e Indexação)
# Acesso a elementos via índices positivos (0 até N-1) e reverso (-1 até -N)
# ================================================================================

# Definição de uma lista com elementos inteiros sequenciais
numeros = [10, 20, 30, 40, 50]

print("--- Acesso por Índices Positivos ---")
print(f"Primeiro elemento [índice 0]: {numeros[0]}")
print(f"Segundo elemento  [índice 1]: {numeros[1]}")
print(f"Terceiro elemento [índice 2]: {numeros[2]}")

print("\n--- Acesso por Índices Negativos (Reverso) ---")
print(f"Último elemento     [índice -1]: {numeros[-1]}")
print(f"Penúltimo elemento  [índice -2]: {numeros[-2]}")
print(f"Primeiro via reverso [índice -5]: {numeros[-5]}")
```

```
--- Acesso por Índices Positivos ---
Primeiro elemento [índice 0]: 10
Segundo elemento  [índice 1]: 20
Terceiro elemento [índice 2]: 30

--- Acesso por Índices Negativos (Reverso) ---
Último elemento     [índice -1]: 50
Penúltimo elemento  [índice -2]: 40
Primeiro via reverso [índice -5]: 10
```

Tópico 3: Manipulação Dinâmica e Percurso em Listas

As listas em Python são dinâmicas e aceitam inserções com o método `.append()`. Para percorrer os elementos, podemos utilizar o percurso **por valor** ou **por posição (índice)**.

```
# ================================================================================
# TÓPICO 3: Manipulação Dinâmica e Percurso em Listas
# Inserção com .append() e iteração por valor vs por posição/índice
# ================================================================================

# Criação de uma lista inicial com três notas
notas = [8.5, 7.0, 9.2]

# Adicionando um novo elemento dinamicamente ao final da lista
notas.append(8.0)
print(f"Lista de notas após adição dinâmica: {notas}\n")

# 1. PERCURSO POR VALOR
# Recomendado quando não precisamos manipular ou saber o índice numérico
print("--- Percurso por Valor (for nota in notas) ---")
for nota in notas:
    print(f"Nota lida: {nota}")

# 2. PERCURSO POR POSIÇÃO/ÍNDICE
# Recomendado quando a posição exata (índice) é necessária para processamento
print("\n--- Percurso por Posição (for i in range(len(notas))) ---")
for i in range(len(notas)):
    print(f"Posição/Índice {i} -> Nota: {notas[i]}")
```

```
Lista de notas após adição dinâmica: [8.5, 7.0, 9.2, 8.0]

--- Percurso por Valor (for nota in notas) ---
Nota lida: 8.5
Nota lida: 7.0
Nota lida: 9.2
Nota lida: 8.0

--- Percurso por Posição (for i in range(len(notas))) ---
Posição/Índice 0 -> Nota: 8.5
Posição/Índice 1 -> Nota: 7.0
Posição/Índice 2 -> Nota: 9.2
Posição/Índice 3 -> Nota: 8.0
```

Tópico 4: A 2ª Dimensão — Matrizes e Loops Aninhados

Uma matriz bidimensional é representada em Python utilizando uma **lista de listas**. A navegação em duas dimensões é realizada através de dois loops `for` aninhados: o externo percorre as **linhas** e o interno as **colunas**.

```
# ================================================================================
# TÓPICO 4: Matrizes (Estruturas Multidimensionais)
# Representação através de listas de listas e varredura com loops aninhados
# ================================================================================

# Definindo uma matriz 3x3
matriz = [
    [1, 2, 3],  # Linha 0
    [4, 5, 6],  # Linha 1
    [7, 8, 9]   # Linha 2
]

# Acesso direto a um elemento específico: matriz[linha][coluna]
# Acessando linha 1, coluna 2 -> elemento de valor 6
print(f"Elemento na posição matriz[1][2]: {matriz[1][2]}\n")

# Varredura completa da matriz com loops aninhados
print("--- Varredura Completa da Matriz (Linhas x Colunas) ---")
for i in range(len(matriz)):          # 1º loop: itera pelas linhas
    for j in range(len(matriz[i])):   # 2º loop: itera pelas colunas da linha atual
        print(f"matriz[{i}][{j}] = {matriz[i][j]}", end=" | ")
    print()  # Quebra de linha ao final de cada linha da matriz
```

```
Elemento na posição matriz[1][2]: 6

--- Varredura Completa da Matriz (Linhas x Colunas) ---
matriz[0][0] = 1 | matriz[0][1] = 2 | matriz[0][2] = 3 |
matriz[1][0] = 4 | matriz[1][1] = 5 | matriz[1][2] = 6 |
matriz[2][0] = 7 | matriz[2][1] = 8 | matriz[2][2] = 9 |
```

Tópico 5: Modelagem de Entidades com Dicionários (`dict`)

Dicionários associam **chaves** (`str`) a **valores** de qualquer tipo (números, strings, listas). Essa estrutura permite modelar objetos e entidades do mundo real com facilidade de acesso através das chaves.

```
# ================================================================================
# TÓPICO 5: Representando Entidades com Dicionários (dict)
# Mapeamento chave -> valor para representação estruturada de dados
# ================================================================================

# Modelando a entidade 'Aluno' usando um dicionário
aluno = {
    "nome": "João",
    "idade": 20,
    "notas": [8.5, 9.0]
}

# Acessando e exibindo os atributos através de suas chaves descritivas
print(f"Nome do Aluno: {aluno['nome']}")
print(f"Idade do Aluno: {aluno['idade']}")
print(f"Notas do Aluno: {aluno['notas']}")

# Operação com estrutura interna (lista de notas dentro do dicionário)
media_aluno = sum(aluno['notas']) / len(aluno['notas'])
print(f"Média calculada: {media_aluno:.2f}")
```

```
Nome do Aluno: João
Idade do Aluno: 20
Notas do Aluno: [8.5, 9.0]
Média calculada: 8.75
```

Tópico 6: Modelagem Avançada com `dataclass`

A abordagem com `@dataclass` aproxima o Python de estruturas fortemente tipadas (*structs*). Ela exige a definição explícita de atributos e seus respectivos tipos, trazendo maior robustez e clareza ao código.

```
# ================================================================================
# TÓPICO 6: Modelagem Avançada com dataclass
# Uso de orientação a objetos com tipagem explícita de atributos
# ================================================================================

from dataclasses import dataclass
from typing import List

# Definição do molde da entidade Aluno usando a anotação @dataclass
@dataclass
class Aluno:
    nome: str
    idade: int
    notas: List[float]

# Instanciando o objeto a partir da classe definida
aluno1 = Aluno(nome="Ana", idade=21, notas=[9.0, 9.5])

# Acesso direto aos atributos utilizando notação por ponto (.atributo)
print(f"Instância de Aluno: {aluno1}")
print(f"Nome do Aluno: {aluno1.nome}")
print(f"Primeira nota: {aluno1.notas[0]}")

# Cálculo da média
media_ana = sum(aluno1.notas) / len(aluno1.notas)
print(f"Média de {aluno1.nome}: {media_ana:.2f}")
```

```
Instância de Aluno: Aluno(nome='Ana', idade=21, notas=[9.0, 9.5])
Nome do Aluno: Ana
Primeira nota: 9.0
Média de Ana: 9.25
```

Tópico 7: Estudo de Caso — Sistema de Cadastro de Alunos

Aplicações Práticas:

1. Inicializar lista mestre vazia `alunos = []`.
2. Cadastrar 3 alunos (nome, idade, matrícula, notas).

3. Exibir os registros cadastrados na tela.
4. Calcular a média aritmética das notas de cada aluno.
5. Identificar via código o aluno com a maior média.

```python
# ============================================================================
# TÓPICO 7: Estudo de Caso — Sistema de Cadastro de Alunos
# Solução Arquitetural: Lista de Dicionários / Objetos
# ============================================================================

# Passo 1: Lista mestre vazia para armazenar todos os alunos cadastrados
alunos = []

# Passo 2: Definindo as entidades individualmente
aluno1 = {
    "nome": "Carlos Silva",
    "idade": 20,
    "matricula": "202401",
    "notas": [7.5, 8.0, 9.0]
}

aluno2 = {
    "nome": "Mariana Souza",
    "idade": 22,
    "matricula": "202402",
    "notas": [9.5, 9.0, 10.0]
}

aluno3 = {
    "nome": "Beatriz Lima",
    "idade": 19,
    "matricula": "202403",
    "notas": [6.0, 7.5, 8.0]
}

# Adicionando os objetos modelados à lista mestre
alunos.append(aluno1)
alunos.append(aluno2)
alunos.append(aluno3)

print("=== PASSO 3 & 4: LISTAGEM DE REGISTROS E CÁLCULO DE MÉDIAS ===\n")

# Variáveis auxiliares para controle do aluno de maior média
maior_media = -1.0
aluno_com_maior_media = None

# Percorrendo a lista principal de alunos
for aluno in alunos:
    nome = aluno["nome"]
    matricula = aluno["matricula"]

    # Isola a lista interna de notas para processamento
    notas_atuais = aluno["notas"]

    # Cálculo da média aritmética do aluno atual
    media = sum(notas_atuais) / len(notas_atuais)

    print(f"Matrícula: {matricula} | Nome: {nome:<15} | Notas: {notas_atuais} | Média: {media:.2f}")

    # Passo 5: Comparação para encontrar a maior média
    if media > maior_media:
        maior_media = media
        aluno_com_maior_media = aluno

print("\n" + "="*60)
print("=== PASSO 5: ALUNO IDENTIFICADO COM A MAIOR MÉDIA ===")
print("="*60)
if aluno_com_maior_media:
    print(f"Nome do Aluno: {aluno_com_maior_media['nome']}")
    print(f"Matrícula:     {aluno_com_maior_media['matricula']}")
    print(f"Maior Média:   {maior_media:.2f}")
```

```
=== PASSO 3 & 4: LISTAGEM DE REGISTROS E CÁLCULO DE MÉDIAS ===

Matrícula: 202401 | Nome: Carlos Silva    | Notas: [7.5, 8.0, 9.0] | Média: 8.17
Matrícula: 202402 | Nome: Mariana Souza   | Notas: [9.5, 9.0, 10.0] | Média: 9.50
Matrícula: 202403 | Nome: Beatriz Lima    | Notas: [6.0, 7.5, 8.0] | Média: 7.17

============================================================
=== PASSO 5: ALUNO IDENTIFICADO COM A MAIOR MÉDIA ===
============================================================
```

```
Nome do Aluno: Mariana Souza
Matrícula:     202402
Maior Média:   9.50
```

Tópico 8: Desafio Prático — Gestão de Estoque

Requisitos do Desafio:

1. Criar entidade **Produto** contendo `nome`, `preco` e `quantidade`.
2. Criar uma lista principal cadastrando **pelo menos 5 produtos**.
3. **Métricas Solicitadas:**

- Calcular o valor total financeiro em estoque (`preco * quantidade`).
- Identificar o produto **mais caro** (maior preço unitário).
- Identificar o produto com **maior volume armazenado** (maior quantidade).

```python
# ================================================================================
# TÓPICO 8: Desafio Prático — Gestão de Estoque
# Lista de produtos com processamento de métricas e busca de extremos
# ================================================================================

# Requisito 1 & 2: Cadastro de pelo menos 5 produtos usando lista de dicionários
estoque = [
    {"nome": "Notebook Pro", "preco": 4500.00, "quantidade": 8},
    {"nome": "Mouse Sem Fio", "preco": 120.00, "quantidade": 45},
    {"nome": "Teclado Mecânico", "preco": 350.00, "quantidade": 25},
    {"nome": "Monitor 27 Ultra", "preco": 1800.00, "quantidade": 12},
    {"nome": "Cadeira Ergonômica", "preco": 1200.00, "quantidade": 15}
]

# Inicialização de variáveis acumuladoras e de referência para os extremos
valor_total_estoque = 0.0
produto_mais_caro = estoque[0]
produto_maior_volume = estoque[0]

print("=== RELATÓRIO DETALHADO DO ESTOQUE CADASTRADO ===\n")

# Processamento dos itens do estoque
for produto in estoque:
    # Cálculo do valor financeiro total do item (Preço * Quantidade)
    subtotal_item = produto["preco"] * produto["quantidade"]

    # Acumula o valor total do estoque
    valor_total_estoque += subtotal_item

    # Impressão formatada dos dados de cada produto
    print(f"Produto: {produto['nome']:<20} | "
          f"Preço Un.: R$ {produto['preco']:8.2f} | "
          f"Qtd: {produto['quantidade']:3d} | "
          f"Subtotal: R$ {subtotal_item:9.2f}")

    # Verificação 1: Produto de maior preço unitário
    if produto["preco"] > produto_mais_caro["preco"]:
        produto_mais_caro = produto

    # Verificação 2: Produto com maior quantidade armazenada
    if produto["quantidade"] > produto_maior_volume["quantidade"]:
        produto_maior_volume = produto

print("\n" + "="*65)
print("=== MÉTRICAS E RESULTADOS DO DESAFIO DE ESTOQUE ===")
print("="*65)
print(f"1. Valor Total Financeiro em Estoque: R$ {valor_total_estoque:,.2f}")
print(f"2. Produto Mais Caro (Preço Un.):    {produto_mais_caro['nome']} (R$ {produto_mais_caro['preco']:.2f})")
print(f"3. Maior Volume de Estoque (Qtd):    {produto_maior_volume['nome']} ({produto_maior_volume['quantidade']} unida
```

```
=== RELATÓRIO DETALHADO DO ESTOQUE CADASTRADO ===

Produto: Notebook Pro         | Preço Un.: R$  4500.00 | Qtd:   8 | Subtotal: R$  36000.00
Produto: Mouse Sem Fio        | Preço Un.: R$   120.00 | Qtd:  45 | Subtotal: R$   5400.00
Produto: Teclado Mecânico     | Preço Un.: R$   350.00 | Qtd:  25 | Subtotal: R$   8750.00
Produto: Monitor 27 Ultra     | Preço Un.: R$  1800.00 | Qtd:  12 | Subtotal: R$  21600.00
Produto: Cadeira Ergonômica   | Preço Un.: R$  1200.00 | Qtd:  15 | Subtotal: R$  18000.00

=================================================================
=== MÉTRICAS E RESULTADOS DO DESAFIO DE ESTOQUE ===
=================================================================
```

```
1. Valor Total Financeiro em Estoque: R$ 89,750.00
2. Produto Mais Caro (Preço Un.):    Notebook Pro (R$ 4500.00)
3. Maior Volume de Estoque (Qtd):    Mouse Sem Fio (45 unidades)
```